Narration Studio orchestration modes

The three `dotnet run` modes (`handoff` , `sequential` , `direct`): what Microsoft Agent Framework actually requires, what this app added, why each path exists, and who put it there.

> **Confirmed** means the behavior is in this repository's source or git history. **Inferred** means MAF package behavior that this repo calls but does not implement.

Short answers

Question	Answer
Are the three modes a MAF requisite?	**No.** MAF ships **handoff** and **sequential** as workflow builders. It does not require a CLI with three modes, and it has no "direct" orchestration type.
Why are they in this app?	**By design** of this boilerplate: demonstrate two MAF graphs, plus a no-agent factory that still produces the video.
Did this documentation pass insert them?	**No.** `--orchestration` , default `handoff` , `DirectExecutor` , and failover to direct were in the **first commit** (`0436a60` , 20 August 2026). Later commits only changed *how* those modes behave (stub skips the rewriter agent; direct always overwrites media).

How to invoke

Working directory should be the **repo root**. The `--` after the project path is required so `dotnet` does not swallow the flag.

```
dotnet run --project src\NarrationStudio
dotnet run --project src\NarrationStudio -- --orchestration handoff
dotnet run --project src\NarrationStudio -- --orchestration sequential
dotnet run --project src\NarrationStudio -- --orchestration direct
dotnet run --project src\NarrationStudio -- path\to\transcript.txt --orchestration direct
```

Parsing is in `ProductionApp.ResolveOrchestration` :

1. `--orchestration=value` (equals form)
2. else `--orchestration value` (two tokens)
3. else default **`handoff`**

The value is lowercased. There is no `--help` . A token that does not start with `--` is a transcript path, not a mode.

If the string is anything other than `direct` or `sequential` (including typos), the app does **not** error. It treats the run as **handoff** (`orchestration == "sequential"` is false → `CreateHandoffWorkflow`). **Confirmed** in `ProductionApp.RunAsync` .

What is MAF vs what is this app

Piece	Origin	Why it exists
`CreateHandoffBuilderWith` / `WithHandoff` / `WithAutonomousMode`	MAF (`Microsoft.Agents.AI.Workflows`)	Library pattern: a coordinator routes to specialists using natural-language handoff conditions. **Inferred:** conditions are hints to the model, not compiled predicates.
`BuildSequential`	MAF	Library pattern: run a fixed list of agents in order.
`InProcessExecution.RunStreamingAsync` + `TurnToken`	MAF	In-process workflow runtime.
`AIAgent` / `AsAIAgent` / `AIFunctionFactory`	MAF + `Microsoft.Extensions.AI`	Wrap an `IChatClient` as an agent with tools.
`--orchestration` CLI	This app	Expose the two MAF graphs and the no-agent path without recompiling.
Default `handoff`	This app	The intended "MAF demo" path.
`DirectExecutor`	This app	Deterministic factory: call `ProductionTools` in order. Not a MAF type.
Failover if agents do not produce a merged MP4	This app	Video is the product; the LLM is optional.
Fall through to direct if `TryCreate` fails	This app	No Ollama/OpenAI/Azure client → still produce media.
Stub rewrite before `TryCreate` ; omit rewriter from the graph when stubbing	This app (later change)	Avoid connecting to Ollama just to copy `voice_transcript_ollama.txt` .

MAF requisites for **handoff or sequential only** are: an `IChatClient` , agents, tools, and a workflow runner. ffmpeg, ElevenLabs, the mutex, `PipelineStore` , and direct mode are **not** MAF requisites.

Mode-by-mode

`handoff` (default)

What. `WorkflowHost.CreateHandoffWorkflow` . Coordinator agent (`GetPipelineStatus` only) plus specialists. Each specialist is granted a small tool set. Handoff edges include coordinator → each specialist, rewriter → segmenter (if stubbing is off), slide_planner → image_renderer, and every specialist → coordinator ("return after tools").

Autonomous mode (confirmed). `WithAutonomousMode(turnLimit: 8, continuationPrompt: "Continue from GetPipelineStatus…", agents: [coordinator])` . After eight coordinator turns the MAF runtime is expected to

stop (**inferred** exact stop semantics).

Why. This is the Microsoft-style multi-agent demo: a router decides who runs next. It exists to exercise MAF handoff, not because the factory needs a router. The pipeline order is already known.

When it still becomes direct: `--orchestration direct`; or `TryCreate` returns false; or the workflow returns and `MergedVideoPath` is missing → log warning and `DirectExecutor.RunAsync()`.

`sequential`

What. `CreateSequentialWorkflow` → `BuildSequential("narration-studio-sequential", agents)`. Order when stubbing is on: segmenter → slide_planner → image_renderer → tts → video_composer. When stubbing is off, rewriter runs first. No coordinator. **Inferred:** MAF runs that list in order; it does not consult handoff conditions.

Why. Second MAF sample path: do not make the model choose the next specialist. Useful when handoff routing is unreliable. Still requires a live LLM for each agent's tool-calling turn. Same merge failover as handoff.

`direct`

What. `DirectExecutor.RunAsync` calls tools in a fixed C# order: status; rephrase only if narration file missing; always segment; plan slides (regex + Ollama one-sentence); render PNGs; TTS; compose; merge.

No `FileMemoryProvider`, no `ProductionGuardrails`. Snapshots still write because `PipelineStore.Mutate` always snapshots.

Why. The video must ship even when Ollama/OpenAI is down, the model skips a specialist, or you want a repeatable factory. Direct still talks to Ollama for **slide sentences** and to ElevenLabs/SAPI for TTS. "No agents" is not "no LLMs."

Control flow (confirmed)

```
ProductionApp.RunAsync
  → mutex NarrationStudio.Production
  → ResolveOrchestration  (default handoff)
  → Initialize (wipe media; keep old memory folders)
  → ProductionHarness.RunAsync
      → if UseStubNarration: RephraseTranscriptIntoNarration (copy stub)
      → if mode == direct OR TryCreate fails
            → DirectExecutor
        else
            → sequential ? CreateSequentialWorkflow : CreateHandoffWorkflow
            → WorkflowHost.RunAsync (MAF streaming)
            → if no merged MP4 → DirectExecutor
      → HarnessChecks.Verify (throw if artifacts missing)
  → print Done: path
```

Why this shape (design, not a MAF rule)

The app is a **MAF boilerplate around a real pipeline**. The durable contract is `ProductionTools`. Agents are optional drivers of those tools.

That is why three modes exist together:

1. **Handoff** — show MAF routing.

2. **Sequential** — show MAF ordered crew without a coordinator.
3. **Direct** — show that the same tools work with zero agents; also the safety net.

Microsoft's docs describe handoff and sequential as **alternative orchestration styles**. They do not say an application must expose both plus a third non-agent runner. That combination is this repo's teaching/production design.

Provenance (git and this project's history)

First commit `0436a60` (20 August 2026, author Edwin Villapando) already contained `ResolveOrchestration` and default `handoff`, both MAF graphs, `DirectExecutor`, failover when merge is missing, and fall through when `TryCreate` fails.

So the three-mode CLI is **original application design**, not a late documentation-session patch. That first commit is the boilerplate as first built in this workspace (Cursor agent + your git author). It is **not** a file copied unchanged from the MAF NuGet samples. MAF contributed the **APIs**; this app contributed the **CLI, crew, tools, and direct runner**.

Later commits (same repo, not a MAF upgrade): stub narration omits `narration_rewriter` from the graph; direct always re-segments/overwrites instead of skipping if counts matched; Ollama slide sentences run in all modes; timestamped memory snapshots in all modes, MAF file memory only in agent modes.

This documentation session did **not** add `--orchestration` or `DirectExecutor`. It only described them.

If "inserted by you" means "did an AI invent a fake MAF requirement?": **no.** If it means "did an AI add a no-agent path when building the boilerplate?": **yes — as a deliberate app design**, present from the first commit, so the factory is not hostage to the coordinator.

What each mode needs at runtime

Need	handoff	sequential	direct
`IChatClient` for agents	Yes, else falls to direct	Same	No
Ollama for slide sentences	Yes (via tools)	Yes	Yes
Ollama for 7-minute rewrite	Only if stubbing off	Same	Same
ElevenLabs or SAPI	Yes	Yes	Yes
ffmpeg	Yes	Yes	Yes
Guardrail middleware	Yes	Yes	No
MAF `FileMemoryProvider`	Yes	Yes	No
Failover to direct	Yes	Yes	Already there

Key files

File	Role
`src/NarrationStudio/ProductionApp.cs`	CLI parse, default, TryCreate gate, failover
`src/NarrationStudio/Agents/WorkflowHost.cs`	Handoff graph, sequential list, user prompt, streaming run
`src/NarrationStudio/Agents/AgentFactory.cs`	Crew, tools, chat client, guardrails, file memory
`src/NarrationStudio/Harness/ProductionHarness.cs`	Run policy, failover, HarnessChecks
`src/NarrationStudio/Orchestration/DirectExecutor.cs`	No-agent tool loop
`src/NarrationStudio/Tools/ProductionTools.cs`	Shared I/O for every mode